

Aula 24 — Sistemas Operacionais Distribuídos: Conceitos e Arquitetura

Disciplina: Sistemas Distribuídos — Uni-FACEF · Prof. Leandro Borges

Abrindo um novo bloco de conteúdo

Com a Aula 23 encerramos o bloco de tolerância a falhas, no qual estudamos como um sistema distribuído continua funcionando — ou falha de forma controlada — quando um de seus componentes para de responder corretamente. A partir de hoje entramos em um novo bloco, dedicado aos sistemas operacionais distribuídos (SODs). As próximas duas aulas vão aprofundar dois problemas específicos desse tema: gerência de processos e escalonamento distribuído (Aula 25) e sistemas de arquivos distribuídos (Aula 26). Esta aula tem um papel diferente: não entrar em detalhe em nenhum desses dois problemas, mas construir o vocabulário e os modelos de arquitetura que vão sustentar as duas aulas seguintes.

O que é um Sistema Operacional Distribuído

Um sistema operacional tradicional gerencia os recursos de uma única máquina: escalona processos na CPU, controla o acesso à memória, organiza o sistema de arquivos em disco e intermedeia o uso de dispositivos. Um Sistema Operacional Distribuído (SOD) tem o mesmo objetivo, mas aplicado a um conjunto de máquinas interligadas por rede: ele estende as funções clássicas de gerência de processos, memória e arquivos para que operem de forma coordenada através de múltiplos computadores, como se esse conjunto fosse uma única máquina virtual grande.

Esse objetivo retoma diretamente o conceito de transparência, apresentado na Semana 2. Transparência, naquele contexto, significava esconder do usuário e do programador os detalhes de que um sistema é composto por várias máquinas separadas — transparência de acesso, de localização, de concorrência, de falhas, entre outras. Um SOD é, nesse sentido, a expressão mais ambiciosa possível desse objetivo: em vez de apenas tornar transparente a comunicação entre processos ou o acesso a um serviço remoto, ele tenta tornar transparente o próprio sistema operacional. Idealmente, um usuário sentado em qualquer terminal do sistema não deveria perceber diferença nenhuma entre rodar um programa localmente ou disparar sua execução em uma máquina remota do mesmo conjunto — o SOD cuidaria de tudo internamente.

SO de Rede (NOS) versus SO Distribuído (DOS)

É importante não confundir um SOD com algo que já usamos no dia a dia: um Sistema Operacional de Rede, ou NOS (do inglês Network Operating System). Nos dois casos existem várias máquinas conectadas por rede, mas a diferença de arquitetura entre eles é fundamental.

Em um NOS, cada máquina roda seu próprio sistema operacional, de forma independente e autônoma — é exatamente o modelo que usamos ao ligar computadores com Windows ou Linux em uma mesma rede local. Sobre esse SO local, adicionamos serviços de rede: compartilhamento de arquivos, impressão remota, autenticação centralizada, acesso remoto via terminal. O usuário de um NOS sabe que existem várias máquinas distintas e precisa, explicitamente, indicar em qual delas quer executar um programa ou de qual delas quer acessar um recurso. A transparência é baixa, mas em compensação cada máquina mantém total autonomia e a administração do sistema é relativamente simples — é o modelo dominante na prática, presente em praticamente toda rede corporativa ou doméstica.

Já em um SO Distribuído, a arquitetura é fundamentalmente diferente: existe uma única camada de software — um kernel distribuído — que gerencia os recursos de todas as máquinas de forma unificada. O usuário não sabe, e não precisa saber, em qual máquina física seus processos estão de fato executando ou onde seus arquivos estão fisicamente armazenados. Decisões como escalonamento de processos, alocação de memória e localização de arquivos são tomadas globalmente pelo sistema, não máquina a máquina. Esse nível de transparência é muito mais alto que o de um NOS, mas o preço é a complexidade: projetar, implementar e manter um kernel verdadeiramente distribuído é uma tarefa muito mais difícil, e por isso SODs puros são raros fora de contextos de pesquisa ou de nichos muito específicos, como clusters de alto desempenho projetados sob medida.

Modelos de arquitetura de um SOD

Quando um SOD (ou uma aproximação dele) é de fato construído, a literatura costuma descrever três modelos de arquitetura para organizar os processadores disponíveis no sistema.

O primeiro é o modelo de estação de trabalho (workstation model). Cada usuário tem sua própria estação de trabalho, com CPU e memória dedicadas — é a configuração mais familiar, presente em laboratórios de informática e escritórios. Como essas máquinas ficam ociosas boa parte do tempo (o usuário está lendo, digitando, pensando, não usando 100% da CPU), o SOD aproveita esse tempo ocioso e faz as estações compartilharem um sistema de arquivos distribuído, de forma que qualquer usuário acesse os mesmos arquivos a partir de qualquer estação. A distribuição, nesse modelo, fica concentrada principalmente no acesso a arquivos e recursos remotos; cada máquina continua sendo dona do seu próprio processamento.

O segundo é o modelo de banco de processadores (processor pool model). Em vez de processadores fixos, dedicados a um usuário específico, existe um conjunto — um pool — de processadores compartilhado, sem dono fixo. Quando um usuário submete uma tarefa, o sistema aloca dinamicamente um ou vários processadores desse pool para executá-la e os libera assim que ela termina, deixando-os disponíveis para a próxima tarefa. Esse modelo tende a usar a capacidade total de processamento de forma muito mais eficiente, já que nenhum processador fica ocioso esperando por um usuário específico — mas em compensação é menos intuitivo, porque não existe mais o conceito de 'minha máquina': o escalonamento de qual tarefa roda em qual processador precisa ser decidido de forma global, considerando carga e disponibilidade em tempo real.

O terceiro é um modelo híbrido, que combina os dois anteriores. Cada usuário continua tendo uma estação de trabalho pessoal para tarefas interativas do dia a dia — editar um documento, navegar, programar — mas o sistema também mantém um pool de processadores compartilhado, reservado para tarefas pesadas ou paralelas, que podem ser submetidas sob demanda quando necessário. Esse modelo tende a ser o equilíbrio mais comum entre desempenho, custo e experiência do usuário, e aparece com frequência em ambientes acadêmicos e corporativos que combinam estações de trabalho comuns com algum tipo de cluster ou servidor de processamento compartilhado.

O papel do middleware

Dado que um SOD puro é difícil de construir e raro na prática, como os sistemas distribuídos que usamos diariamente — de sistemas de arquivos em nuvem a plataformas de e-commerce — conseguem entregar boa parte dessa mesma sensação de transparência? A resposta, na grande maioria dos casos, é o middleware.

Middleware é uma camada de software que roda sobre sistemas operacionais convencionais e independentes — cada máquina do sistema continua rodando seu próprio Linux, Windows ou outro SO local, exatamente como em um NOS

— mas que simula, para as aplicações que rodam por cima dela, boa parte da transparência que um SOD verdadeiro ofereceria. Mecanismos de RPC e RMI, sistemas de mensageria, serviços de coordenação e bibliotecas de comunicação distribuída são exemplos de middleware: eles escondem dos programadores de aplicação os detalhes de que existem várias máquinas separadas, sem exigir que o kernel de cada máquina seja substituído por um kernel distribuído único.

É exatamente por isso que, na prática, middleware — não SOD puro — é a abordagem dominante em sistemas distribuídos reais. Construir e manter um middleware é uma tarefa muito mais tratável do que construir um kernel distribuído do zero, e ainda assim entrega, para a grande maioria das aplicações, um nível de transparência suficiente. Vale reter essa distinção: o SOD é o conceito arquitetural mais puro e ambicioso; o middleware é a solução de engenharia que a indústria efetivamente adota para se aproximar desse ideal.

Desafios específicos de projeto de um SOD

Independentemente de estarmos falando de um SOD puro ou de uma aproximação via middleware, projetar esse tipo de sistema levanta desafios que não existem em um sistema operacional de máquina única.

O primeiro é a gerência global de recursos: decidir onde alocar CPU, memória e arquivos não é mais uma questão local, resolvida dentro de uma única máquina, mas uma decisão que precisa considerar o estado de todas as máquinas do sistema simultaneamente. O segundo é o escalonamento distribuído: escolher em qual nó cada processo deve rodar, balanceando a carga entre máquinas, sem o luxo de uma visão centralizada e de baixíssima latência que um escalonador de SO tradicional tem sobre os processos de uma única máquina. O terceiro é a comunicação entre kernels: em um SO tradicional, componentes internos do kernel se comunicam através de chamadas de função extremamente rápidas e confiáveis; em um SOD, essa comunicação precisa atravessar a rede, que é ordens de magnitude mais lenta e sujeita a atrasos, perdas de mensagem e partições — exatamente os problemas de comunicação e falha que já estudamos em blocos anteriores da disciplina.

Esses três desafios — gerência global de recursos, escalonamento distribuído e comunicação entre kernels — são o pano de fundo que vamos aprofundar nas próximas duas aulas: a Aula 25 entra em detalhe em como um SOD decide onde cada processo executa, e a Aula 26 examina como um SOD organiza um sistema de arquivos que parece único, mesmo estando fisicamente espalhado por várias máquinas.

Referências

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems: Principles and Paradigms. 3. ed. Pearson, 2017. Cap. 1 (Introdução) e Cap. 3 (Processos).

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. Distributed Systems: Concepts and Design. 5. ed. Addison-Wesley, 2011. Cap. 1 e 2.

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems. 4. ed. 2023 (edição digital gratuita).